

Shapley values and extensions

Definition of Shapley effects

General definition

- ▶ Arise from game theory ; fair share of a player in a cooperative game
- ▶ A k -player game with the set of players $\mathcal{K} := \{1, \dots, k\}$ is defined as a real-valued function that maps a subset of $\mathcal{K}$ to its corresponding cost c
- ▶ The quantity $c(J)$ represents the cost that arises when the players in the subset J participate in the game

Definition of Shapley effects

General definition

- ▶ The Shapley value $\nu_i[c]$ of player i with respect to the cost function $c(\cdot)$ is defined as

$$\nu_i[c] := \sum_{I \subset K \setminus \{i\}} \frac{(k - |I| - 1)! \cdot |I|!}{k!} (c(I \cup \{i\}) - c(I))$$

- ▶ $Sh_i[c]$ is the incremental cost of including player i in set I averaged over all sets I .
- ▶ Note that the sum contains **an exponential number** of terms

Definition of Shapley effects

Shapley values and global SA

In the context of global sensitivity analysis,

- ▶ Set of players : set of inputs of the model
- ▶ Choice of the cost function c ?

Definition of Shapley effects

Shapley values and global SA

Natural requirements for the cost function

- ▶ For each J , $c(J)$ measures the variance of Y caused by the uncertainty of the inputs in J
- ▶ $c(\emptyset) = 0$ and $c(K) = \text{Var}[Y]$.
- ▶ Some possible choices
 - ▶ Choice 1

$$c_1(I) = \text{Var}(\mathbb{E}[Y|X_I])$$

- ▶ Choice 2

$$c_2(I) = \mathbb{E}[\text{Var}(Y|X_{-I})]$$

Definition of Shapley effects

Theorem

The Shapley values defined using cost function c_1 and c_2 are equivalent.

Shapley effect of the i -th input

Shapley value Sh_i obtained by applying the cost function c_1 or c_2 to the general definition

$$Sh_i := \nu_i[c_1] = \nu_i[c_2]$$

Definition of Shapley effects

Main property

Even if there is dependence or structural interactions among the inputs, one has

$$\sum_i Sh_i = Var(Y)$$

It is not the case for Sobol indices!

Estimating the Shapley effect

Two main questions

Chen, Covert, Lundberg & Lee, 2023) organize the literature around **two independent axes**:

- ▶ how “absent” features are handled (the *coalitional game*)
- ▶ how the exponential exact sum is made tractable (the *estimation strategy*).

Estimating the Shapley effect

Feature removal approaches

- ▶ **Baseline Shapley values** — absent features replaced with a fixed reference x^b . Exactly computable, but the baseline choice is arbitrary.
- ▶ **Marginal (interventional) Shapley values** — absent features replaced by draws from their marginal distribution. Easiest to estimate; the most common variant.
- ▶ **Conditional Shapley values** — absent features drawn from the conditional distribution given observed features. Faithful to correlated features, but hard to estimate (empirical matching, parametric/Gaussian assumptions, generative or surrogate models).

Estimating the Shapley effect

Tractable estimation strategies in the model-agnostic case

The approaches are based on several equivalent mathematical characterizations

- ▶ The semivalue (SV) approach : Weighted average of marginal contributions over subsets sampled from a weighting distribution (ApproSemivalue; L-Shapley / C-Shapley)
- ▶ The random-order (RO) approach : Average marginal contribution across random permutations (IME; ApproShapley / permutation sampling)
- ▶ Shapley value as solution to a weighted least-squares regression : KernelSHAP, FastSHAP

Estimating the Shapley effect

Tractable estimation strategies in the model-specific case

In the case we know the black-box model, model-specific methods exploit its structure

- ▶ Linear models (LinearSHAP, Correlated LinearSHAP). In this case, the Shapley value has a closed-form solution which can be used to avoid sampling
- ▶ Tree models (Interventional TreeSHAP, Path-dependent TreeSHAP). A decision tree is a sequence of binary splits, that can be exploited, via a clever recursive algorithm, to compute the exact Shapley value in polynomial time
- ▶ Deep models (DeepLIFT, DeepSHAP, DASP, ShapNet). Each of these reuses the network's own forward/backward computational graph

Details about estimation in the model-agnostic case

Overview of the semi-value and random order approaches

- ▶ Both approaches are exact mathematical formulas for the *same* Shapley value
- ▶ They are just two different ways of writing the same sum,
- ▶ That difference in bookkeeping is what each sampling scheme exploits.

Details about estimation in the model-agnostic case

Semivalue characterization.

Sum over **subsets** S directly, weighted by how likely that subset size is under the classical Shapley weighting:

$$Sh_i(v) = \sum_{S \subseteq D \setminus \{i\}} P(S)(v(S \cup \{i\}) - v(S)), \quad P(S) = \frac{|S|! (|D| - |S| - 1)!}{|D|!}$$

Details about estimation in the model-agnostic case

Random-order characterization.

- ▶ Sum over **permutations** π (orderings of all players)
- ▶ For each one look at who came before player i :

$$Sh_i(v) = \frac{1}{|D|!} \sum_{\pi \in \Pi(D)} (v(Pre^i(\pi) \cup \{i\}) - v(Pre^i(\pi)))$$

where $Pre^i(\pi)$ is the set of players appearing before i in ordering π .

Details about estimation in the model-agnostic case

Semivalue sampling (ApproSemivalue): sample subsets, one player at a time

- ▶ To estimate ϕ_i : repeatedly draw random subsets $S \subseteq D \setminus \{i\}$ (with probability $P(S)$), compute $v(S \cup \{i\}) - v(S)$ each time, and average.
- ▶ This is done **independently for each player**. Estimating ϕ_1 tells you nothing directly about ϕ_2 — a entirely separate batch of subsets would be drawn for player 2.

Details about estimation in the model-agnostic case

Random-order sampling (ApproShapley): sample permutations, all players at once

- ▶ Draw one random permutation π of *all* players.
- ▶ Walk along it: start with $S = \emptyset$, and as each player is added one at a time, record their marginal contribution at that point in the walk.
- ▶ **One permutation gives a marginal contribution estimate for every player simultaneously** — for each player, one simply reads off what got added when they entered the ordering.

Details about estimation in the model-agnostic case

Song, Nelson & Staum (2016): estimating Shapley effects

- ▶ Apply the **random-order** (permutation) approach directly to the **variance-based** cost function from earlier:

$$v(S) = c_1(S) = \text{Var}(\mathbb{E}[Y \mid X_S])$$

- ▶ Same permutation-walk machinery as ApproShapley — sample a permutation π , walk along it, record the marginal contribution of each player as it enters
- ▶ The only thing that changes versus generic random-order sampling: *what $v(S)$ means* — here it is a variance of a conditional expectation, not a single model prediction

Details about estimation in the model-agnostic case

Why Song's approach is expensive

- ▶ Unlike KernelSHAP's $v(S) = \mathbb{E}[f(x_S, X^{(-S)})]$ (one prediction, one model call), $v(S) = \text{Var}(\mathbb{E}[Y | X_S])$ **cannot** be evaluated with a single query
- ▶ Estimating it needs its own **nested Monte Carlo** procedure at *every step* of the permutation walk:
 - ▶ **Outer loop** — sample values of X_S
 - ▶ **Inner loop** — for each fixed X_S , resample X_{-S} to estimate the inner conditional expectation $\mathbb{E}[Y | X_S]$
 - ▶ Take the variance of the inner estimates across the outer draws
- ▶ Cost compounds: (many sampled permutations) $\times$ (a full nested double loop **per step, per permutation**)

Estimating the Shapley effect

Limits of Song's approaches

- ▶ Algorithm with a high computational cost
- ▶ Alternatives : consider surrogate models
- ▶ Leads to **LIME** and **KernelSHAP**

LIME

LIME (2016)

- ▶ Local surrogate models : interpretable models that are used to explain **individual predictions** of black box machine learning models.
- ▶ Local interpretable model-agnostic explanations (LIME) : approach for fitting surrogate models.
- ▶ Surrogate models trained to approximate the predictions of the underlying black box model.

LIME

LIME (2016)

- ▶ Explanation model for a given instance $\mathbf{x}$?
- ▶ Starting point
 - ▶ black-box model $\hat{f}$ trained from observations $\mathcal{D} := \{(x_i, y_i), i = 1, \dots, n\}$
 - ▶ white box/surrogate model $\hat{f}_{surrogate}$
- ▶ Local surrogate models with interpretability constraint : solution of the following optimization problem

$$\hat{f}_{surrogate}(\mathbf{x}) = \arg \min_{g \in G} L(\hat{f}, g, \pi_{\mathbf{x}}) + \Omega(g)$$

where L loss, Ω complexity measure

LIME

LIME (2016)

The recipe for training local surrogate models:

- ▶ Select your instance of interest for which you want to have an explanation of its black box prediction.
- ▶ Perturb your dataset and get the black box predictions for these new points.
- ▶ Weight the new samples according to their proximity to the instance of interest.
- ▶ Train a weighted, interpretable model on the dataset with the variations.
- ▶ Explain the prediction by interpreting the local model.

LIME

LIME (2016)

- ▶ Linear regression can be chosen as an interpretable surrogate model.
- ▶ Number of features K to select
- ▶ Several methods for **training models with low complexity**, for e.g. Lasso.

KernelSHAP

KernelSHAP (2021)

- ▶ KernelSHAP : Linear LIME + Shapley values
- ▶ SHAP approach consider a weighted linear regression model as the local surrogate model and an appropriate weighting kernel without any penalization
- ▶ The regression coefficients of the corresponding LIME surrogate model estimates the SHAP values.
- ▶ A blog about KernelSHAP
<https://iancovert.com/blog/kernelshap/>

KernelSHAP

KernelSHAP (2021)

- Set for a given instance x and a given model f

$$v(S) = \mathbb{E}[f(x_S, X^{(-S)})]$$

- ▶ Search for coefficients that minimize

$$\hat{\beta} = \arg \min_{\beta} \sum_{S \subseteq D} W(S) (u(S) - v(S))^2,$$

- ▶ $u(S) = \beta_\emptyset + \sum_{i \in S} \beta_i$ and $W(S) = \frac{|D| - 1}{\binom{|D|}{|S|} |S| (|D| - |S|)}$
- ▶ The approximation of the Shapley values are $\phi_1, \dots, \phi_d$, $\phi_\emptyset$ is the output of the model without any features

KernelSHAP

Why this is not just a heuristic

- ▶ This is a **fourth exact characterization** of the Shapley value — solved *exactly* over all $2^{|D|}$ coalitions, $\hat{\beta}_i = \phi_i(v)$, with no approximation error
- ▶ Sits alongside the semivalue, random-order, and multilinear-extension characterizations: same $\phi_i(v)$, a different equivalent formula
- ▶ $W(S)$ is **not** an arbitrary kernel choice: it is derived from the same combinatorial weighting $P(S) = \frac{|S|! (|D| - |S| - 1)!}{|D|!}$ that appears in the semivalue formula — the “Shapley kernel” is that weighting wearing a different hat

KernelSHAP

A subtlety: the $S = \emptyset$ and $S = D$ singularity

- ▶ $W(S) = \frac{|D| - 1}{(|D| - |S|)|S|}$ is **undefined** at $S = \emptyset$ and $S = D$ (the denominator vanishes)
- ▶ Fix: treat $v(\emptyset) = \phi_\emptyset$ and $v(D)$ as **known, fixed** points, not part of the fit, and solve a **reduced regression** over the remaining $2^{|D|} - 2$ coalitions
- ▶ This is exactly what forces the **efficiency axiom** to hold: $\sum_i \phi_i = v(D) - v(\emptyset)$ by construction, not by luck

KernelSHAP

KernelSHAP

In practice to avoid exponential computational complexity

- ▶ The WLS problem above is **exact**; what is approximated is *how it is solved*: instead of summing over all $2^{|D|}$ coalitions, sample a moderate number of subsets according to $W(S)$
- ▶ Solve the (reduced) WLS problem on this sample instead of the full enumeration
- ▶ Implemented in the SHAP library

FastSHAP

FastSHAP (2022)

- ▶ KernelSHAP solves a **fresh WLS problem for every new instance $\mathbf{x}$** — accurate, but slow when many predictions need explaining (e.g. a live dashboard, auditing a whole test set)
- ▶ FastSHAP's idea: **amortize** this cost by training a single **explainer network $\phi_\theta(\mathbf{x})$** once, offline
- ▶ At inference time: **one forward pass** of ϕ_θ replaces solving a WLS problem from scratch

FastSHAP

Training objective

- ▶ Same weighted least-squares loss as KernelSHAP, but β is replaced by the **output of a network** evaluated at $\mathbf{x}$:

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathbf{x}} \mathbb{E}_{S \sim W(S)} \left[\left(\phi_{\theta}(\mathbf{x})^{\top} z_S + v_{\mathbf{x}}(\emptyset) - v_{\mathbf{x}}(S) \right)^2 \right]$$

where $z_S \in \{0, 1\}^{|D|}$ is the membership indicator of S

- ▶ The efficiency constraint $\sum_i \phi_{\theta}(\mathbf{x})_i = v_{\mathbf{x}}(D) - v_{\mathbf{x}}(\emptyset)$ is enforced at inference (additive normalization), not solved coalition-by-coalition
- ▶ Training samples many $(\mathbf{x}, S)$ pairs **once**; the network then generalizes to new, unseen $\mathbf{x}$

FastSHAP

FastSHAP vs. KernelSHAP

- ▶ **Speed:** KernelSHAP — many model queries *per instance*; FastSHAP — one forward pass per instance, after a one-time training cost
- ▶ **Exactness:** KernelSHAP converges to the exact WLS solution as the sample size grows; FastSHAP is only as good as the trained network — no convergence guarantee to the true $\phi_i(v)$
- ▶ **A structural difference:** KernelSHAP cleanly separates “estimate $\hat{v}(S)$ ” from “solve for ϕ ”; FastSHAP **fuses** the two — the network is trained end-to-end directly against the WLS objective, so there is no separate, swappable $\hat{v}$ stage
- ▶ **Bottom line:** FastSHAP trades exactness/guarantees for amortized, real-time explanation — the right tool when the same model must explain *many* instances, not a strictly better version of KernelSHAP